

Data Structures and Algorithms

Think clearly about problems, then solve them
efficiently

Sample study notes · Knowledge Dungeon

Contents

Algorithms and Complexity	3
Arrays and Strings	11
Linked Lists	20
Stacks and Queues	29
Recursion and Backtracking	38
Searching and Sorting	48
Trees and Binary Search Trees	58
Heaps and Hash Tables	69
Graphs	79
Greedy Algorithms and Dynamic Programming	89

Algorithms and Complexity

A **data structure** is a way of organising data so that it can be used efficiently. An **algorithm** is a finite, step-by-step procedure for solving a problem. Together they are the heart of programming: the right data structure makes an algorithm simple, and the right algorithm makes a program fast. This chapter explains how we measure "fast" and "memory-hungry" in a way that does not depend on any particular computer.

What Makes a Good Algorithm?

An algorithm should be:

- **Correct:** it produces the right output for every valid input.
- **Finite:** it always finishes.
- **Definite:** every step is precise and unambiguous.
- **Efficient:** it uses as little time and memory as reasonably possible.

Correctness comes first. A fast algorithm that gives wrong answers is useless. But among correct algorithms, efficiency often decides whether a program finishes in a second or in a week.

Why Not Just Time It?

We could run two programs and use a stopwatch, but the result depends on the computer, the language, the compiler, other running programs and the particular input. We need a measure that describes how the running time **grows as the input grows**. That measure is **asymptotic complexity**.

Counting Steps

Consider finding the largest number in a list.

```
def find_max(nums):  
    best = nums[0]           # 1 step  
    for x in nums:           # n iterations  
        if x > best:         # 1 comparison each  
            best = x         # at most 1 assignment each  
    return best              # 1 step
```

For a list of n numbers, this does roughly $2n + 2$ basic steps. As n becomes large, the constants 2 and +2 stop mattering. What matters is that the work grows **in proportion to n** . We say the algorithm runs in **$O(n)$** time, read "order n " or "big-O of n ".

Big-O, Big-Omega and Big-Theta

- **Big-O (O)** gives an **upper bound**: the algorithm grows *no faster than* this. $f(n) = O(g(n))$ if $f(n) \leq c * g(n)$ for some constant c and all large enough n .
- **Big-Omega** gives a **lower bound**: the algorithm grows *at least* this fast.

- **Big-Theta** gives a **tight bound**: both upper and lower, so the growth rate is exactly this.

In everyday use, people say "big-O" and usually mean the tight bound for the worst case.

Rules for Simplifying

1. **Drop constants:** $O(3n)$ becomes $O(n)$; $O(n / 2)$ becomes $O(n)$.
2. **Keep only the dominant term:** $O(n^2 + 5n + 100)$ becomes $O(n^2)$, because for large n the n^2 term swamps the rest.
3. **Different inputs get different variables:** looping over list A and then list B is $O(a + b)$, not $O(n)$.

Common Complexity Classes

Big-O	Name	Example	n = 1,000,000 needs about
$O(1)$	Constant	Access an array element by index	1 step
$O(\log n)$	Logarithmic	Binary search	20 steps
$O(n)$	Linear	Scan a list once	1 million steps
$O(n \log n)$	Linearithmic	Merge sort	20 million steps
$O(n^2)$	Quadratic	Compare every pair	10^{12} steps
$O(2^n)$	Exponential	Try every subset	Impossible
$O(n!)$	Factorial	Try every ordering	Impossible

Key idea: A computer doing around a hundred million simple steps per second finishes an $O(n \log n)$ job on a million items instantly, but an $O(n^2)$ job takes hours. Choosing a better complexity class beats buying a faster computer.

Why $\log n$ Is So Small

$\log_2(n)$ asks "how many times can I halve n before reaching 1?" Halving a million takes only about 20 steps; halving a billion takes about 30. Any algorithm that throws away half the remaining problem at each step, such as binary search, runs in $O(\log n)$.

Analysing Loops

- A single loop over n items: **$O(n)$** .
- Two nested loops, each over n items: **$O(n^2)$** .

```
for i in range(n):
    for j in range(n):
        print(i, j)           # runs n * n times
```

- Nested loops where the inner loop shrinks:

```
for i in range(n):
    for j in range(i + 1, n):
        compare(i, j)         # runs n(n-1)/2 times, still
                                 $O(n^2)$ 
```

- A loop that doubles or halves its counter: **$O(\log n)$** .

```
i = 1
while i < n:
    i *= 2                                # runs about log2(n) times
```

- Consecutive (not nested) loops add up: $O(n) + O(n) = O(n)$.

Best, Worst and Average Case

The same algorithm can behave differently on different inputs of the same size. Consider searching a list for a value from the start:

- **Best case:** the value is first. $O(1)$.
- **Worst case:** the value is last or absent. $O(n)$.
- **Average case:** on average we look through half the list. $O(n)$.

We usually quote the **worst case**, because it is a guarantee: the algorithm will never be slower than this.

Space Complexity

Space complexity measures the extra memory an algorithm needs as a function of input size. `find_max` uses only a couple of variables, so it needs $O(1)$ extra space. An algorithm that copies the input into a new list uses $O(n)$. Recursive algorithms also use stack space for each pending call (Chapter 5).

There is often a **time-space trade-off**: using extra memory, such as a hash table, can make an algorithm much faster.

Amortised Analysis

Some operations are usually cheap but occasionally expensive. Appending to a Python list is normally $O(1)$, but when the underlying array is full, the list allocates a bigger array and copies everything, which is $O(n)$. Because the array doubles in size each time, these expensive copies are rare. Averaged over many appends, each append costs $O(1)$. This average over a sequence of operations is called **amortised** complexity.

A Worked Comparison: Checking for Duplicates

Problem: does a list contain any duplicate values?

Approach 1: compare every pair.

```
def has_duplicate_slow(nums):
    for i in range(len(nums)):
        for j in range(i + 1, len(nums)):
            if nums[i] == nums[j]:
                return True
    return False
```

Time $O(n^2)$, space $O(1)$.

Approach 2: sort first. Duplicates end up next to each other.

```
def has_duplicate_sort(nums):
    nums = sorted(nums)
    return any(nums[i] == nums[i + 1] for i in
               range(len(nums) - 1))
```

Time $O(n \log n)$, space $O(n)$ for the sorted copy.

Approach 3: use a set.

```
def has_duplicate_fast(nums):
    seen = set()
    for x in nums:
        if x in seen:
            return True
        seen.add(x)
    return False
```

Time $O(n)$ on average, space $O(n)$. This is the time-space trade-off in action: extra memory buys a big speed-up.

Abstract Data Types

An **abstract data type (ADT)** describes *what* operations a structure supports, not *how* they are implemented. A **stack** ADT promises push, pop and peek. It could be implemented with an array or a linked list. Separating the interface from the implementation lets us change the implementation without changing the programs that use it.

The rest of this book studies the most important data structures (arrays, linked lists, stacks, queues, trees, heaps, hash tables and graphs) and the algorithms that go with them.

Summary

- Algorithms must be correct, finite, definite and efficient.

- Asymptotic complexity describes how time or memory grows with input size, independent of hardware.
- Big-O is an upper bound; drop constants and lower-order terms.
- Common classes, from fastest to slowest: $O(1)$, $O(\log n)$, $O(n)$, $O(n \log n)$, $O(n^2)$, $O(2^n)$, $O(n!)$.
- Nested loops multiply; consecutive loops add; halving loops give $O(\log n)$.
- Worst-case analysis gives guarantees; amortised analysis averages over sequences of operations.
- Extra memory can often buy speed.

Practice Questions

1. Simplify: $O(5n^2 + 3n \log n + 20)$.
2. What is the time complexity of a loop that runs while $i < n$, with i starting at n and being halved each time?
3. Give an example of an algorithm whose best and worst cases differ.
4. Explain amortised $O(1)$ append with an example.
5. Write an $O(n)$ algorithm to find whether two numbers in a list add up to a target value.

Arrays and Strings

The array is the simplest and most important data structure. It stores elements side by side in memory, which makes reading any element by its position instant. Strings are, at heart, arrays of characters. This chapter covers how arrays work, their costs, and the classic techniques used to solve array and string problems: two pointers, sliding windows and prefix sums.

How Arrays Work

An **array** stores elements of the same type in **contiguous** (adjacent) memory locations. If the array starts at memory address `base` and each element takes `size` bytes, element `i` lives at:

```
address(i) = base + i * size
```

Because this is simple arithmetic, accessing any element by its **index** takes $O(1)$ time, no matter how large the array is. This is called **random access**.

Indexes usually start at 0, so an array of `n` elements has indexes 0 to `n - 1`.

Static and Dynamic Arrays

- A **static array** has a fixed size chosen when it is created, as in C: `int marks[50];`.

- A **dynamic array** grows automatically. Python's `list`, Java's `ArrayList` and C++'s `vector` are dynamic arrays. When full, they allocate a larger block (often double the size) and copy the elements across, which gives amortised $O(1)$ appends (Chapter 1).

Costs of Array Operations

Operation	Time	Why
Access by index	$O(1)$	Address arithmetic
Update by index	$O(1)$	Address arithmetic
Search (unsorted)	$O(n)$	May check every element
Search (sorted)	$O(\log n)$	Binary search
Append at end	$O(1)$ amortised	Occasional resize
Insert at position i	$O(n)$	Shift later elements right
Delete at position i	$O(n)$	Shift later elements left

Key idea: Arrays are excellent for reading and for adding at the end, but inserting or deleting in the middle is slow, because elements must shift to keep the memory contiguous.

Traversal and Basic Operations

```
marks = [72, 85, 91, 64, 88]

total = 0
for m in marks:                # traversal: O(n)
    total += m
average = total / len(marks)

marks.append(79)                # O(1) amortised
marks.insert(0, 50)             # O(n): everything shifts right
marks.pop()                     # O(1): remove last
marks.pop(0)                    # O(n): everything shifts left
```

Reversing an Array In Place

```
def reverse(a):
    left, right = 0, len(a) - 1
    while left < right:
        a[left], a[right] = a[right], a[left]
        left += 1
        right -= 1
```

This runs in $O(n)$ time and $O(1)$ extra space. It is also our first example of the **two-pointer technique**.

Two-Dimensional Arrays

A **2-D array** (matrix) stores data in rows and columns, such as a chessboard, an image or a spreadsheet. In memory it is still one long block. In **row-major order** (used by C and Python's NumPy by default), row 0 comes first, then row 1, and so on:

```
address(r, c) = base + (r * number_of_columns + c) * size
```

Traversing row by row is faster than column by column, because it reads memory in order and uses the CPU cache well.

Technique 1: Two Pointers

Use two indexes that move through the array according to some rule, often from both ends towards the middle.

Problem: in a *sorted* array, find two numbers that add up to a target.

```
def pair_with_sum(a, target):
    left, right = 0, len(a) - 1
    while left < right:
        s = a[left] + a[right]
        if s == target:
            return left, right
        if s < target:
            left += 1           # need a bigger sum
        else:
            right -= 1         # need a smaller sum
    return None
```

The brute-force approach checks all pairs in $O(n^2)$. Two pointers solve it in $O(n)$, because each step rules out an element forever.

Other two-pointer problems include removing duplicates from a sorted array, merging two sorted arrays, and checking whether a string is a palindrome.

Technique 2: Sliding Window

A **sliding window** is a range [left, right] that moves across the array, growing on one side and shrinking on the other. It turns many $O(n^2)$ "check every subarray" problems into $O(n)$.

Problem: the maximum sum of any k consecutive elements.

```
def max_window_sum(a, k):
    window = sum(a[:k])
    best = window
    for right in range(k, len(a)):
        window += a[right] - a[right - k]    # add new
        element, drop old one
        best = max(best, window)
    return best
```

Instead of re-adding k numbers for every window, we update the sum in $O(1)$ as the window slides.

Variable-size windows grow until a condition breaks, then shrink from the left. For example, finding the longest substring without repeating characters:

```
def longest_unique(s):
    seen = {}
    left = best = 0
    for right, ch in enumerate(s):
        if ch in seen and seen[ch] >= left:
            left = seen[ch] + 1    # jump past the
            previous copy
        seen[ch] = right
        best = max(best, right - left + 1)
    return best
```

Technique 3: Prefix Sums

A **prefix sum** array stores running totals: $\text{prefix}[i]$ is the sum of the first i elements.

```
def build_prefix(a):
    prefix = [0]
    for x in a:
        prefix.append(prefix[-1] + x)
    return prefix

# sum of a[i..j] inclusive, in O(1):
# prefix[j + 1] - prefix[i]
```

After $O(n)$ preparation, any range sum takes $O(1)$. This is invaluable when there are many range queries on data that does not change.

Kadane's Algorithm

Problem: find the contiguous subarray with the largest sum (the array may contain negatives).

```
def max_subarray(a):
    best = current = a[0]
    for x in a[1:]:
        current = max(x, current + x)    # extend the run,
        # or start fresh
        best = max(best, current)
    return best
```

At each element we decide whether to extend the current run or start a new one. It runs in $O(n)$ with $O(1)$ space, a classic example of a simple, powerful idea.

Strings

A **string** is a sequence of characters. In many languages, including Python and Java, strings are **immutable**: they cannot be changed after creation. "Modifying" a string really creates a new one.

Key idea: Building a string by repeated `s = s + piece` in a loop can take $O(n^2)$ time, because each step copies the whole string. Collect pieces in a list and use `"".join(pieces)` instead, which is $O(n)$.

Common String Operations

Operation	Python	Typical cost
Length	<code>len(s)</code>	$O(1)$
Character at i	<code>s[i]</code>	$O(1)$
Substring	<code>s[i:j]</code>	$O(j - i)$
Search for pattern	<code>s.find(p)</code>	$O(n * m)$ worst case
Concatenate	<code>s + t</code>	$O(\text{len}(s) + \text{len}(t))$
Split / join	<code>s.split(),</code> <code>"".join(parts)</code>	$O(n)$

Classic String Problems

Palindrome check (two pointers):

```
def is_palindrome(s):
    s = [c.lower() for c in s if c.isalnum()]
    return s == s[::-1]
```

Anagram check (counting):

```
from collections import Counter

def are_anagrams(a, b):
    return Counter(a) == Counter(b)        # "listen" and
    "silent" -> True
```

Counting characters with a dictionary or a fixed-size array of 26 counts is a pattern that appears in countless string problems.

Pattern Matching

The naive way to find a pattern of length m in a text of length n tries every starting position: $O(n * m)$ in the worst case. Smarter algorithms such as **KMP** (Knuth-Morris-Pratt) and **Rabin-Karp** achieve roughly $O(n + m)$ by avoiding repeated comparisons. KMP precomputes how far the pattern can safely shift after a mismatch; Rabin-Karp compares hash values of windows before comparing characters.

Summary

- Arrays store elements contiguously, giving $O(1)$ access by index but $O(n)$ insertion and deletion in the middle.
- Dynamic arrays grow by resizing, with amortised $O(1)$ appends.
- Two pointers, sliding windows and prefix sums turn many $O(n^2)$ problems into $O(n)$.

- Kadane's algorithm finds the maximum subarray sum in $O(n)$.
- Strings are often immutable; build them with join, not repeated concatenation.
- Counting characters solves many string problems; KMP and Rabin-Karp speed up pattern matching.

Practice Questions

1. Why is inserting at the start of an array $O(n)$?
2. Use two pointers to remove duplicates from a sorted array in place.
3. Find the maximum average of any 4 consecutive elements using a sliding window.
4. Using prefix sums, answer: what is the sum of elements from index 2 to 5 of [3, 1, 4, 1, 5, 9, 2]?
5. Trace Kadane's algorithm on [-2, 1, -3, 4, -1, 2, 1, -5, 4].

Linked Lists

Arrays keep elements side by side, which makes access fast but insertion and deletion slow. A **linked list** takes the opposite approach: each element lives in its own **node**, anywhere in memory, and points to the next node. Inserting or removing a node only means changing a few pointers. This chapter covers singly, doubly and circular linked lists, and the classic techniques for working with them.

Nodes and Pointers

A node holds two things: the **data** and a **reference** (pointer) to the next node.

```
class Node:
    def __init__(self, data):
        self.data = data
        self.next = None
```

A list is identified by its **head**, the first node. The last node's next is None, marking the end.

```
head -> [10 | *] -> [20 | *] -> [30 | *] -> None
```

Singly Linked List Operations

Traversal

```
def print_list(head):  
    node = head  
    while node:  
        print(node.data, end=" -> ")  
        node = node.next  
    print("None")
```

To reach the k -th element we must walk from the head, so access by position is **$O(n)$** . There is no random access.

Insert at the Head

```
def push_front(head, data):  
    node = Node(data)  
    node.next = head  
    return node                # the new head
```

This is **$O(1)$** : no shifting, unlike an array.

Insert After a Given Node

```
def insert_after(prev, data):  
    node = Node(data)  
    node.next = prev.next  
    prev.next = node
```

Also $O(1)$, *if* we already have a reference to `prev`. Finding it may take $O(n)$.

Key idea: Always connect the new node to the rest of the list *before* redirecting the previous node. If you do it in the wrong order, you lose the reference to the rest of the list.

Insert at the Tail

Without a tail pointer we must walk to the end: $O(n)$. Keeping a separate `tail` reference makes it $O(1)$.

Delete a Node by Value

```
def delete(head, value):
    if head and head.data == value:
        return head.next          # deleting the head
    node = head
    while node and node.next:
        if node.next.data == value:
            node.next = node.next.next  # bypass the node
            return head
        node = node.next
    return head
```

The deleted node is skipped over; in Python the garbage collector frees it. In C you must free it yourself.

Arrays vs Linked Lists

Operation	Array	Singly linked list
Access k-th element	$O(1)$	$O(n)$
Insert / delete at front	$O(n)$	$O(1)$
Insert / delete at end	$O(1)$ amortised	$O(1)$ with tail pointer (delete needs $O(n)$)
Insert / delete in middle (given position)	$O(n)$	$O(1)$ after reaching it
Extra memory per element	None	One pointer (or two)
Cache friendliness	Excellent	Poor (nodes scattered)

In practice, arrays are faster for most workloads because modern CPUs love contiguous memory. Linked lists win when you frequently insert or delete at known positions, and they are the building blocks of other structures such as stacks, queues, hash-table chains and adjacency lists.

Doubly Linked Lists

Each node in a **doubly linked list** has pointers to both the next and the previous node.

```
None <- [10] <-> [20] <-> [30] -> None
```

```
class DNode:
    def __init__(self, data):
        self.data = data
        self.prev = None
        self.next = None
```

Advantages:

- Traverse in both directions.
- Delete a node in $O(1)$ given only a reference to it, because we can reach its predecessor directly.

The cost is an extra pointer per node and more pointers to update on every insertion or deletion. Browser history (back and forward) and LRU caches are classic uses.

Sentinel (Dummy) Nodes

Special cases for empty lists and for the head and tail make linked-list code error-prone. A **sentinel** is a dummy node that always sits before the first real node (and often after the last). With sentinels, every real node has a predecessor and a successor, so insertion and deletion need no special cases.

Circular Linked Lists

In a **circular** linked list, the last node points back to the first instead of `None`. It is useful for round-robin scheduling, where a CPU cycles through processes endlessly, and for multiplayer turn order. When traversing, stop when you return to the starting node, or you will loop forever.

Classic Linked List Techniques

Reversing a List

```
def reverse(head):
    prev = None
    node = head
    while node:
        nxt = node.next      # remember the rest
        node.next = prev     # reverse the pointer
        prev = node          # move prev forward
        node = nxt           # move node forward
    return prev              # new head
```

$O(n)$ time, $O(1)$ space. Tracing this on paper with three nodes is the best way to understand pointer manipulation.

Fast and Slow Pointers

Two pointers moving at different speeds solve several problems elegantly.

Find the middle node: the slow pointer moves one step, the fast pointer two. When fast reaches the end, slow is in the middle.

```
def middle(head):
    slow = fast = head
    while fast and fast.next:
        slow = slow.next
        fast = fast.next.next
    return slow
```

Detect a cycle (Floyd's algorithm): if the list has a loop, the fast pointer eventually laps the slow one and they meet. If fast

reaches None, there is no cycle.

```
def has_cycle(head):
    slow = fast = head
    while fast and fast.next:
        slow = slow.next
        fast = fast.next.next
    if slow is fast:
        return True
    return False
```

Both run in $O(n)$ time and $O(1)$ space.

Key idea: "Tortoise and hare" pointers find middles, cycles and the k-th node from the end without extra memory.

k-th Node from the End

Move one pointer k steps ahead, then move both together.

When the leading pointer reaches the end, the trailing pointer is k nodes from the end.

Merging Two Sorted Lists

```
def merge(a, b):
    dummy = tail = Node(0)
    while a and b:
        if a.data <= b.data:
            tail.next, a = a, a.next
        else:
            tail.next, b = b, b.next
        tail = tail.next
    tail.next = a or b
    return dummy.next
```

The dummy node avoids special-casing the first element. This merge step is the core of merge sort on linked lists.

Common Mistakes

- **Losing the head:** moving head itself while traversing. Use a separate variable.
- **Null dereference:** accessing `node.next.data` when `node.next` is `None`. Check first.
- **Forgetting to update the head** after inserting or deleting at the front.
- **Infinite loops** in circular lists or after accidentally creating a cycle.

Summary

- A linked list stores data in nodes connected by pointers, starting from the head.
- Access by position is $O(n)$; insertion and deletion at a known node are $O(1)$.
- Doubly linked lists allow backward traversal and $O(1)$ deletion of a known node.
- Circular lists loop back to the start; sentinel nodes remove special cases.

- Key techniques: iterative reversal, fast and slow pointers (middle, cycle detection, k-th from end) and merging sorted lists.
- Arrays are usually faster in practice, but linked lists underpin stacks, queues, hash chains and graphs.

Practice Questions

1. Why is access by index $O(n)$ in a linked list but $O(1)$ in an array?
2. Trace the reverse function on the list 1 -> 2 -> 3.
3. Explain why Floyd's cycle detection algorithm always finds a cycle if one exists.
4. Write a function to delete the k-th node from the end in one pass.
5. When would you choose a doubly linked list over a singly linked list?

Stacks and Queues

Stacks and queues are restricted lists: you can only add and remove elements at specific ends. That restriction is exactly what makes them useful. A **stack** gives you the most recent item first, and a **queue** gives you the oldest item first. Between them, they power undo buttons, function calls, expression evaluation, printers, schedulers and breadth-first search.

Stacks: Last In, First Out

A **stack** follows **LIFO** order: the last element added is the first one removed. Think of a stack of plates: you add to the top and take from the top.

Core operations, all $O(1)$:

- `push(x)`: add `x` to the top.
- `pop()`: remove and return the top element.
- `peek()` / `top()`: look at the top element without removing it.
- `is_empty()`: check whether the stack has no elements.

Popping an empty stack is called **underflow**; pushing onto a full fixed-size stack is **overflow**.

Implementations

With a dynamic array (a Python list), the end of the list is the top:

```
stack = []
stack.append(10)      # push
stack.append(20)
top = stack[-1]       # peek -> 20
item = stack.pop()    # pop  -> 20
```

With a linked list, the head is the top, since inserting and deleting at the head is $O(1)$.

```
class Stack:
    def __init__(self):
        self.head = None
    def push(self, x):
        node = Node(x)
        node.next = self.head
        self.head = node
    def pop(self):
        if self.head is None:
            raise IndexError("pop from empty stack")
        x = self.head.data
        self.head = self.head.next
        return x
```

Applications of Stacks

The Call Stack

When a function calls another function, the computer pushes a **stack frame** holding the return address and local variables. When the called function returns, its frame is popped. This is why very deep recursion causes a **stack overflow** error (Chapter 5).

Undo and Browser Back

Every action is pushed onto a stack. Undo pops the most recent one. A second stack can hold undone actions to support redo.

Balanced Brackets

Check whether brackets in an expression like `{[()()]}` are balanced.

```
def balanced(expr):
    pairs = {"(": ")", "[": "]", "{": "}"
    stack = []
    for ch in expr:
        if ch in "([{":
            stack.append(ch)
        elif ch in pairs:
            if not stack or stack.pop() != pairs[ch]:
                return False
    return not stack
```

Every closing bracket must match the most recent unmatched opening bracket, which is exactly LIFO. Compilers and code editors use this idea.

Expression Notation and Evaluation

We normally write **infix** expressions: $3 + 4 * 2$. Computers find **postfix** (Reverse Polish) notation easier, because it needs no brackets or precedence rules: $3\ 4\ 2\ *\ +$.

Infix	Prefix	Postfix
A + B	+ A B	A B +
A + B * C	+ A * B C	A B C * +
(A + B) * C	* + A B C	A B + C *

Evaluating postfix with a stack: scan left to right; push numbers; when you see an operator, pop two numbers, apply the operator, and push the result.

```
def eval_postfix(tokens):
    stack = []
    for t in tokens:
        if t in "+-* /":
            b, a = stack.pop(), stack.pop()
            stack.append({"+": a + b, "-": a - b, "*": a * b,
                          "/": a / b}[t])
        else:
            stack.append(float(t))
    return stack.pop()

eval_postfix("3 4 2 * +".split())    # 11.0
```

Note the order: the first value popped is the *right* operand.

Infix to postfix conversion uses the **shunting-yard algorithm**: output operands immediately, push operators onto a stack, and pop operators of higher or equal precedence before

pushing a new one. Opening brackets are pushed; a closing bracket pops operators until the matching opening bracket.

Monotonic Stacks

A **monotonic stack** keeps its elements in increasing or decreasing order. It solves "next greater element" problems in $O(n)$:

```
def next_greater(a):
    result = [-1] * len(a)
    stack = []                # indexes waiting for a
    greater element
    for i, x in enumerate(a):
        while stack and a[stack[-1]] < x:
            result[stack.pop()] = x
        stack.append(i)
    return result

next_greater([4, 5, 2, 25])    # [5, 25, 25, -1]
```

Each index is pushed and popped at most once, so the total work is $O(n)$, even though there is a loop inside a loop.

Queues: First In, First Out

A **queue** follows **FIFO** order: the first element added is the first removed, like a line at a ticket counter.

Core operations, all **$O(1)$** with a good implementation:

- `enqueue(x)`: add `x` at the **rear**.
- `dequeue()`: remove and return the element at the **front**.

- `front()`: look at the front element.
- `is_empty()`.

Implementations

A plain Python list is a poor queue, because `list.pop(0)` shifts every element and is $O(n)$. Use `collections.deque`, a double-ended queue with $O(1)$ operations at both ends:

```
from collections import deque

q = deque()
q.append("A")           # enqueue
q.append("B")
first = q.popleft()     # dequeue -> "A"
```

A linked list with both head and tail pointers also works: enqueue at the tail, dequeue at the head.

Circular Queues

With a fixed-size array, dequeuing from the front leaves unused space behind. A **circular queue** (ring buffer) wraps the rear index back to the start of the array:

```
rear = (rear + 1) % capacity
front = (front + 1) % capacity
```

Keeping a count of elements distinguishes a full queue from an empty one. Ring buffers are used for streaming data, keyboard buffers and network packets.

Key idea: The modulo operator lets a fixed array behave like an endless loop, so no space is wasted and nothing needs shifting.

Variations

- **Deque (double-ended queue):** insert and remove at both ends. It can act as both a stack and a queue. Deques also power the **sliding window maximum** technique.
- **Priority queue:** elements leave in order of **priority**, not arrival. An emergency room treats the most critical patient first. Priority queues are usually implemented with heaps (Chapter 8).

Applications of Queues

- **Scheduling:** operating systems queue processes for the CPU; printers queue jobs.
- **Breadth-first search:** explores a graph level by level using a queue (Chapter 9).
- **Buffers:** data arriving faster than it can be processed waits in a queue, as with video streaming.
- **Message queues:** in large systems, services communicate by placing messages on queues, such as orders waiting to be processed.

Stack or Queue?

Question	Use
Need the most recent item first?	Stack
Need to process in arrival order?	Queue
Need to backtrack (undo, depth-first search)?	Stack
Need to explore level by level (shortest path in steps)?	Queue
Need the most important item first?	Priority queue

Building One from the Other

A classic exercise: implement a queue using two stacks. Push onto an "in" stack. To dequeue, if the "out" stack is empty, pop everything from "in" onto "out" (which reverses the order), then pop from "out". Each element moves at most twice, so operations are amortised $O(1)$.

Summary

- Stacks are LIFO with $O(1)$ push, pop and peek; queues are FIFO with $O(1)$ enqueue and dequeue.
- Stacks power function calls, undo, bracket matching, expression evaluation and monotonic-stack problems.
- Postfix expressions are evaluated with a stack; the shunting-yard algorithm converts infix to postfix.

- Use `collections.deque` for queues in Python; circular queues reuse a fixed array with modulo arithmetic.
- Deques allow both ends; priority queues serve the highest priority first.
- Queues power scheduling, buffering and breadth-first search.

Practice Questions

1. Convert $(A + B) * (C - D)$ to postfix and prefix.
2. Evaluate the postfix expression $5\ 1\ 2\ +\ 4\ *\ +\ 3\ -$ step by step.
3. Why is `list.pop(0)` a bad way to dequeue in Python?
4. Implement a queue using two stacks and explain its amortised cost.
5. Use a monotonic stack to find the next greater element for $[2, 7, 3, 5, 4, 6, 8]$.

Recursion and Backtracking

A function that calls itself sounds like a trick, but recursion is one of the most natural ways to solve problems that contain smaller copies of themselves. Folders contain folders, trees contain subtrees, and sorting a list can mean sorting two halves. This chapter explains how recursion works, how to write it correctly, and how **backtracking** uses recursion to explore choices systematically.

The Idea

A **recursive** function solves a problem by:

1. handling a **base case**, a version small enough to answer directly, and
2. reducing larger inputs to **smaller instances of the same problem**, solving those recursively, and combining the results.

The classic example is the factorial: $n! = n \times (n - 1)!$, with $0! = 1$.

```
def factorial(n):  
    if n == 0:                # base case  
        return 1  
    return n * factorial(n - 1) # recursive case
```

Key idea: Every recursive call must move closer to a base case. Without a base case, or without progress towards it, the function calls itself forever.

How Recursion Runs: The Call Stack

Each call gets its own **stack frame** holding its parameters and local variables. Calls pile up until a base case is reached, then return one by one.

```
factorial(3)
  = 3 * factorial(2)
    = 2 * factorial(1)
      = 1 * factorial(0)
        = 1
      = 1
    = 2
  = 6
```

At the deepest point, four frames are on the stack. The recursion uses $O(n)$ stack space. If the depth is too large, the program crashes with a **stack overflow**. Python's default limit is about 1,000 frames.

Thinking Recursively

A reliable way to design recursive functions is the "leap of faith":

1. **Define** exactly what the function returns for an input.
2. **Assume** it already works correctly for smaller inputs.

3. **Use** that assumption to solve the current input.
4. **Handle** the base case(s).

Example: the sum of a list.

```
def total(nums):
    if not nums:
        return 0
    return nums[0] + total(nums[1:])  # first element + sum
    of the rest
```

(Slicing copies the list, so this particular version is $O(n^2)$; passing an index instead avoids the copies. It is shown here for clarity.)

Reversing a String

```
def reverse(s):
    if len(s) <= 1:
        return s
    return reverse(s[1:]) + s[0]
```

Power in $O(\log n)$

Computing x^n by multiplying n times is $O(n)$. Recursion does better by halving:

```
def power(x, n):
    if n == 0:
        return 1
    half = power(x, n // 2)
    return half * half if n % 2 == 0 else half * half * x
```

Each call halves n , so it takes $O(\log n)$ multiplications. This is **fast exponentiation**.

Recurrence Relations

The running time of a recursive function can be written as a **recurrence**:

- factorial: $T(n) = T(n - 1) + O(1)$, giving $O(n)$.
- binary search: $T(n) = T(n / 2) + O(1)$, giving $O(\log n)$.
- merge sort: $T(n) = 2T(n / 2) + O(n)$, giving $O(n \log n)$.

The **Master Theorem** solves recurrences of the form $T(n) = aT(n / b) + O(n^d)$ by comparing d with $\log_b(a)$:

- if $d > \log_b(a)$: $O(n^d)$
- if $d = \log_b(a)$: $O(n^d \log n)$
- if $d < \log_b(a)$: $O(n^{\log_b a})$

For merge sort, $a = 2$, $b = 2$ and $d = 1$, so $d = \log_2(2) = 1$, which gives $O(n \log n)$.

The Danger of Repeated Work

The Fibonacci sequence is 0, 1, 1, 2, 3, 5, 8, ... where each number is the sum of the previous two.

```
def fib(n):
    if n < 2:
        return n
    return fib(n - 1) + fib(n - 2)
```

This is correct but terribly slow. `fib(5)` calls `fib(4)` and `fib(3)`; `fib(4)` calls `fib(3)` again, and so on. The number of calls grows roughly as $O(2^n)$. `fib(50)` would take hours.

The fix is **memoization**: remember answers you have already computed.

```
from functools import lru_cache

@lru_cache(maxsize=None)
def fib(n):
    if n < 2:
        return n
    return fib(n - 1) + fib(n - 2)
```

Now each value is computed once: $O(n)$ time. This idea grows into **dynamic programming** in Chapter 10.

Types of Recursion

- **Linear recursion**: one recursive call per invocation (factorial).
- **Tree recursion**: several calls per invocation (naive Fibonacci), which often explodes in cost.

- **Tail recursion:** the recursive call is the very last action, so the frame could be reused. Some languages optimise this into a loop; Python does not.
- **Indirect recursion:** A calls B, which calls A.

Recursion vs Iteration

Anything recursive can be written with loops and an explicit stack, and vice versa.

	Recursion	Iteration
Readability	Natural for trees and divide-and-conquer	Natural for simple repetition
Memory	Uses call-stack frames	Usually constant extra memory
Risk	Stack overflow on deep inputs	Infinite loops
Speed	Function-call overhead	Usually slightly faster

Use recursion where it makes the solution clearer, especially for trees, graphs and divide-and-conquer, and prefer loops for simple linear repetition.

Divide and Conquer

Divide and conquer splits a problem into independent subproblems, solves them recursively, and combines the results.

1. **Divide** the input into smaller parts.
2. **Conquer** each part recursively.

3. **Combine** the partial answers.

Merge sort and quick sort (Chapter 6) and binary search are the classic examples.

Backtracking

Backtracking builds a solution step by step, and **undoes** (backtracks) a choice as soon as it cannot lead to a valid solution. It explores a tree of choices depth-first.

The general pattern:

```
def backtrack(state):
    if is_solution(state):
        record(state)
        return
    for choice in choices(state):
        if is_valid(state, choice):
            make(state, choice)
            backtrack(state)
            undo(state, choice)          # the "backtrack" step
```

Example: All Subsets

```
def subsets(nums):
    result, current = [], []
    def go(i):
        if i == len(nums):
            result.append(current[:])
            return
        current.append(nums[i])      # choose nums[i]
        go(i + 1)
        current.pop()                # un-choose it
        go(i + 1)
    go(0)
    return result

subsets([1, 2, 3])    # 8 subsets
```

There are 2^n subsets, so any algorithm that lists them all takes at least $O(2^n)$ time.

Example: Permutations

```
def permutations(nums):
    result = []
    def go(current, remaining):
        if not remaining:
            result.append(current)
            return
        for i in range(len(remaining)):
            go(current + [remaining[i]], remaining[:i] +
                remaining[i + 1:])
    go([], nums)
    return result
```

There are $n!$ permutations.

Example: N-Queens

Place N queens on an $N \times N$ chessboard so that no two attack each other. Place one queen per row; for each row, try each column; skip columns and diagonals that are already attacked; if a row has no safe column, backtrack to the previous row and move that queen.

Key idea: Backtracking's power comes from **pruning**: abandoning a partial solution the moment it breaks a rule, which skips huge parts of the search tree.

Sudoku solvers, crossword fillers and maze solvers all use backtracking.

Summary

- Recursion solves a problem through smaller instances of itself, with a base case to stop.
- Each call uses a stack frame; deep recursion can overflow the stack.
- Design recursively with the leap of faith: define, assume, use, handle the base case.
- Recurrences describe recursive running times; the Master Theorem solves common ones.
- Tree recursion can repeat work exponentially; memoization fixes it.

- Divide and conquer splits, solves and combines; backtracking explores choices and undoes bad ones, pruning early.

Practice Questions

1. Write a recursive function to count the digits in a positive integer.
2. Draw the call tree of the naive $\text{fib}(5)$. How many calls are made?
3. Use the Master Theorem to solve $T(n) = 4T(n/2) + O(n)$.
4. Write a backtracking function that prints all binary strings of length n with no two consecutive 1s.
5. Explain how pruning makes backtracking faster than brute force, using N-Queens.

Searching and Sorting

Searching and sorting are the bread and butter of computing. Every time you look up a contact, filter products by price or see a leaderboard, some searching or sorting algorithm is at work. This chapter covers linear and binary search, the simple quadratic sorts, the efficient $O(n \log n)$ sorts, and sorts that beat $O(n \log n)$ by avoiding comparisons.

Linear Search

Check each element in turn until you find the target.

```
def linear_search(a, target):
    for i, x in enumerate(a):
        if x == target:
            return i
    return -1
```

Time $O(n)$, space $O(1)$. It works on any list, sorted or not, and is the right choice for small or unsorted data.

Binary Search

If the array is **sorted**, we can do far better. Compare the target with the middle element; if the target is smaller, it can only be in the left half, otherwise the right half. Each step halves the search space.

```
def binary_search(a, target):
    lo, hi = 0, len(a) - 1
    while lo <= hi:
        mid = (lo + hi) // 2
        if a[mid] == target:
            return mid
        if a[mid] < target:
            lo = mid + 1
        else:
            hi = mid - 1
    return -1
```

Time **$O(\log n)$** : a million sorted items need at most about 20 comparisons.

Key idea: Binary search is famous for off-by-one bugs. Decide whether your range is inclusive $[lo, hi]$ or half-open $[lo, hi)$, and keep the loop condition and updates consistent with that choice.

Variations

- **First or last occurrence** of a repeated value: when you find a match, record it and keep searching to the left (or right).
- **Lower bound:** the first index whose value is $\geq$ target. It is the position where the target would be inserted. Python's `bisect_left` does this.

- **Binary search on the answer:** when a yes/no condition changes only once as a number increases ("can we finish all tasks in X hours?"), binary search over X instead of over an array.

Properties of Sorting Algorithms

- **Time complexity** (best, average, worst).
- **Space: in-place** algorithms use $O(1)$ or $O(\log n)$ extra memory.
- **Stability:** a **stable** sort keeps equal elements in their original relative order. If students are already sorted by name and you then stably sort by marks, students with equal marks stay alphabetical.
- **Adaptivity:** runs faster on nearly sorted input.

Simple Quadratic Sorts

These are easy to understand and fine for tiny arrays, but $O(n^2)$ makes them impractical for large data.

Bubble Sort

Repeatedly swap adjacent elements that are out of order. After each pass, the largest remaining element "bubbles" to the end.

```
def bubble_sort(a):
    n = len(a)
    for i in range(n - 1):
        swapped = False
        for j in range(n - 1 - i):
            if a[j] > a[j + 1]:
                a[j], a[j + 1] = a[j + 1], a[j]
                swapped = True
        if not swapped:           # already sorted: stop early
            break
```

Worst $O(n^2)$, best $O(n)$ with the early exit. Stable and in place.

Selection Sort

Find the smallest element and swap it into the first position; then the smallest of the rest into the second position; and so on. Always $O(n^2)$ comparisons, but at most $n - 1$ swaps, which helps when writing to memory is expensive. Not stable.

Insertion Sort

Build a sorted section at the front. Take each new element and slide it left into its correct place, like sorting a hand of playing cards.

```
def insertion_sort(a):
    for i in range(1, len(a)):
        key = a[i]
        j = i - 1
        while j >= 0 and a[j] > key:
            a[j + 1] = a[j]
            j -= 1
        a[j + 1] = key
```

Worst $O(n^2)$, but **$O(n)$ on nearly sorted data**. It is stable, in place and very fast for small arrays, which is why real-world sorts switch to insertion sort for tiny pieces.

Merge Sort

Merge sort is divide and conquer:

1. Split the array into two halves.
2. Recursively sort each half.
3. **Merge** the two sorted halves into one sorted array.

```
def merge_sort(a):
    if len(a) <= 1:
        return a
    mid = len(a) // 2
    left, right = merge_sort(a[:mid]), merge_sort(a[mid:])
    merged, i, j = [], 0, 0
    while i < len(left) and j < len(right):
        if left[i] <= right[j]:          # <= keeps it
            stable
            merged.append(left[i]); i += 1
        else:
            merged.append(right[j]); j += 1
    return merged + left[i:] + right[j:]
```

The array is halved $\log n$ times, and each level of merging does $O(n)$ work, so merge sort is **$O(n \log n)$ in every case**. It is **stable** but needs **$O(n)$ extra space**. It is also ideal for linked lists and for **external sorting** of data too large for memory.

Quick Sort

Quick sort is also divide and conquer, but the hard work happens *before* the recursion:

1. Pick a **pivot** element.
2. **Partition** the array so that elements smaller than the pivot come before it and larger ones after it. The pivot is now in its final position.
3. Recursively sort the two sides.

```
def quick_sort(a, lo=0, hi=None):
    if hi is None:
        hi = len(a) - 1
    if lo < hi:
        p = partition(a, lo, hi)
        quick_sort(a, lo, p - 1)
        quick_sort(a, p + 1, hi)

def partition(a, lo, hi):          # Lomuto scheme, last
    element as pivot
    pivot, i = a[hi], lo
    for j in range(lo, hi):
        if a[j] < pivot:
            a[i], a[j] = a[j], a[i]
            i += 1
    a[i], a[hi] = a[hi], a[i]
    return i
```

- **Average case:** $O(n \log n)$, with small constants, so it is often the fastest sort in practice.

- **Worst case:** $O(n^2)$, when the pivot is always the smallest or largest element, for example choosing the last element of an already sorted array.
- In place ($O(\log n)$ stack space on average), but **not stable**.

Key idea: Choosing the pivot at random, or as the median of the first, middle and last elements, makes the worst case extremely unlikely.

Heap Sort

Heap sort builds a max-heap (Chapter 8) from the array, then repeatedly swaps the maximum to the end and restores the heap. It is $O(n \log n)$ in every case and in place, but not stable, and it is usually slower in practice than quick sort because of poor cache behaviour.

Comparing the Main Sorts

Algorithm	Best	Average	Worst	Space	Stable
Bubble	$O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$	Yes
Selection	$O(n^2)$	$O(n^2)$	$O(n^2)$	$O(1)$	No
Insertion	$O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$	Yes
Merge	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	$O(n)$	Yes
Quick	$O(n \log n)$	$O(n \log n)$	$O(n^2)$	$O(\log n)$	No
Heap	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	$O(1)$	No

The Comparison Lower Bound

Any sort that works only by comparing pairs of elements needs **at least $O(n \log n)$** comparisons in the worst case. There are $n!$ possible orderings, and each comparison at best halves the possibilities, so about $\log_2(n!)$ comparisons, roughly $n \log n$, are needed. Merge sort and heap sort are therefore optimal among comparison sorts.

Non-Comparison Sorts

If we know more about the keys, we can beat $n \log n$.

Counting Sort

For integers in a small range 0 to k , count how many times each value occurs, then write them out in order. Time $O(n + k)$. Excellent for marks out of 100 or ages, useless if k is huge.

Radix Sort

Sort numbers digit by digit, from the least significant digit to the most, using a stable counting sort for each digit. Time $O(d \times (n + b))$ for d digits in base b . It is great for fixed-length keys such as phone numbers or IDs.

Bucket Sort

Spread values that are uniformly distributed (say between 0 and 1) into n buckets, sort each small bucket, and concatenate. Average $O(n)$.

Sorting in Real Life

Library sorts are hybrids tuned for real data:

- **Timsort** (Python's `sorted` and Java's `object sort`) combines merge sort and insertion sort, detects already sorted runs, and is stable. It is $O(n)$ on sorted data and $O(n \log n)$ in the worst case.
- **Introsort** (C++ `std::sort`) starts with quick sort, switches to heap sort if the recursion gets too deep, and uses insertion sort for small pieces.

```
students = [("Riya", 88), ("Arjun", 92), ("Meera", 88)]
students.sort(key=lambda s: s[1], reverse=True)      # stable:
    Riya stays before Meera
```

Summary

- Linear search is $O(n)$ on any data; binary search is $O(\log n)$ on sorted data.
- Binary search extends to first/last occurrence, lower bounds and searching over answers.
- Bubble, selection and insertion sort are $O(n^2)$; insertion sort is excellent on small or nearly sorted data.
- Merge sort is a stable, guaranteed $O(n \log n)$ sort using $O(n)$ space; quick sort is usually fastest but has an $O(n^2)$ worst case.

- Comparison sorts cannot beat $O(n \log n)$; counting, radix and bucket sort can when keys are suitable.
- Real libraries use hybrids like Timsort and Introsort.

Practice Questions

1. Trace binary search for 23 in [2, 5, 8, 12, 16, 23, 38, 56, 72, 91].
2. What does it mean for a sort to be stable? Give a situation where stability matters.
3. Show the passes of insertion sort on [5, 2, 4, 6, 1, 3].
4. Why does quick sort degrade to $O(n^2)$ on a sorted array with the last element as pivot? How do you prevent it?
5. When would counting sort be a poor choice?

Trees and Binary Search Trees

Arrays, linked lists, stacks and queues are all **linear**: each element has one successor. Many kinds of data are **hierarchical** instead: a company's reporting structure, the folders on your computer, the structure of an HTML page, or a family tree. **Trees** represent hierarchy naturally, and **binary search trees** add ordering to allow fast searching, insertion and deletion.

Tree Vocabulary

A **tree** is a collection of **nodes** connected by **edges**, with no cycles, and with one special node called the **root**.

- **Parent / child:** a node directly above / below another.
- **Siblings:** nodes with the same parent.
- **Leaf:** a node with no children.
- **Internal node:** a node with at least one child.
- **Subtree:** a node together with all its descendants.
- **Depth** of a node: the number of edges from the root to it. The root has depth 0.

- **Height** of a tree: the number of edges on the longest path from the root to a leaf.
- **Degree** of a node: its number of children.

A tree with n nodes always has exactly $n - 1$ edges.

Binary Trees

In a **binary tree**, each node has at most two children, called **left** and **right**.

```
class TreeNode:
    def __init__(self, val):
        self.val = val
        self.left = None
        self.right = None
```

Special kinds of binary trees:

- **Full**: every node has 0 or 2 children.
- **Complete**: every level is filled except possibly the last, which is filled from left to right. Heaps use complete trees (Chapter 8).
- **Perfect**: all internal nodes have two children and all leaves are at the same depth. A perfect tree of height h has $2^{h+1} - 1$ nodes.
- **Balanced**: the heights of the left and right subtrees of every node differ by a small amount, keeping the height about $\log n$.

- **Degenerate (skewed):** every node has one child, so the tree is really a linked list.

Key idea: The height of a tree controls the speed of most tree operations. A balanced tree with a million nodes is only about 20 levels deep; a skewed one is a million levels deep.

Tree Traversals

To **traverse** a tree is to visit every node once. There are two families.

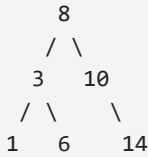
Depth-First Traversals

These go deep along a branch before backtracking. They differ only in *when* the node itself is visited.

- **Preorder** (node, left, right): useful for copying a tree or printing folder structures.
- **Inorder** (left, node, right): on a binary search tree, visits values in **sorted order**.
- **Postorder** (left, right, node): useful for deleting a tree or computing folder sizes, where children must be finished first.

```
def inorder(node, out):
    if node:
        inorder(node.left, out)
        out.append(node.val)
        inorder(node.right, out)
```

For this tree:



- Preorder: 8, 3, 1, 6, 10, 14
- Inorder: 1, 3, 6, 8, 10, 14
- Postorder: 1, 6, 3, 14, 10, 8

Breadth-First (Level-Order) Traversal

Visit nodes level by level, left to right, using a queue.

```
from collections import deque

def level_order(root):
    if not root:
        return []
    out, q = [], deque([root])
    while q:
        node = q.popleft()
        out.append(node.val)
        if node.left:
            q.append(node.left)
        if node.right:
            q.append(node.right)
    return out
```

Level order for the tree above: 8, 3, 10, 1, 6, 14.

All traversals take $O(n)$ time. Recursive depth-first traversals use $O(h)$ stack space, where h is the height.

Recursive Thinking with Trees

Trees are recursive by nature: a tree is a root plus two smaller trees. Many problems become short recursive functions.

```
def height(node):
    if node is None:
        return -1
    # an empty tree has height -1
    return 1 + max(height(node.left), height(node.right))

def count_nodes(node):
    if node is None:
        return 0
    return 1 + count_nodes(node.left) + count_nodes(node.right)
```

Binary Search Trees (BST)

A **binary search tree** is a binary tree with an ordering rule:

- every value in a node's **left** subtree is **smaller** than the node, and
- every value in its **right** subtree is **larger**.

The example tree above is a BST.

Searching

Start at the root. If the target is smaller, go left; if larger, go right; stop when you find it or reach an empty spot.

```
def search(node, target):
    while node and node.val != target:
        node = node.left if target < node.val else node.right
    return node
```

This is binary search on a tree: $O(h)$, where h is the height.

Insertion

Search for the value; when you fall off the tree, attach a new node there.

```
def insert(node, val):
    if node is None:
        return TreeNode(val)
    if val < node.val:
        node.left = insert(node.left, val)
    elif val > node.val:
        node.right = insert(node.right, val)
    return node
```

Deletion

Deletion has three cases:

1. **Leaf:** simply remove it.
2. **One child:** replace the node with its child.

3. **Two children:** replace the node's value with its **inorder successor** (the smallest value in its right subtree), then delete that successor, which has at most one child.

Other Useful Operations

- **Minimum:** keep going left. **Maximum:** keep going right.
- **Sorted output:** inorder traversal.
- **Range queries:** visit only subtrees that can contain values in the range.

Validating a BST

A common mistake is to check only that each node's children are on the correct side. The rule applies to *entire subtrees*, so pass down allowed bounds:

```
def is_bst(node, lo=float("-inf"), hi=float("inf")):
    if node is None:
        return True
    if not (lo < node.val < hi):
        return False
    return is_bst(node.left, lo, node.val) and
           is_bst(node.right, node.val, hi)
```

The Balance Problem

BST operations cost $O(h)$. If values are inserted in random order, the height is about $\log n$ on average. But inserting already sorted values (1, 2, 3, 4, ...) builds a skewed tree where every node only has a right child, so $h = n$ and operations become $O(n)$.

Self-balancing BSTs fix this by restructuring the tree after insertions and deletions.

AVL Trees

An **AVL tree** keeps, at every node, a **balance factor** (height of left subtree minus height of right subtree) of -1, 0 or +1. When an insertion or deletion breaks this, the tree performs **rotations**:

- **Left-Left case:** single right rotation.
- **Right-Right case:** single left rotation.
- **Left-Right case:** left rotation on the child, then right rotation on the node.
- **Right-Left case:** right rotation on the child, then left rotation on the node.

A rotation is a local rearrangement of three nodes that preserves the BST ordering while reducing the height.

Red-Black Trees

A **red-black tree** colours nodes red or black and enforces rules (for example, no two reds in a row, and every path from a node to its leaves has the same number of blacks). It is a little less strictly balanced than AVL but needs fewer rotations on updates. Java's `TreeMap` and C++'s `std::map` use red-black trees.

Operation	Unbalanced BST (worst)	Balanced BST (AVL, red-black)
Search	$O(n)$	$O(\log n)$
Insert	$O(n)$	$O(\log n)$
Delete	$O(n)$	$O(\log n)$

Other Important Trees

- **B-trees and B+ trees:** multi-way trees with many keys per node, designed for disks. Databases and file systems use them for indexes.
- **Tries (prefix trees):** each edge represents a character, so words sharing a prefix share a path. They power autocomplete and spell checkers, and look up a word in $O(\text{length of word})$.
- **Segment trees and Fenwick trees:** answer range queries (sums, minimums) with updates in $O(\log n)$.
- **Expression trees:** internal nodes are operators and leaves are operands; postorder traversal evaluates them.

Summary

- Trees represent hierarchy: a root, parents, children, leaves, depth and height.
- Binary trees have at most two children per node; complete and balanced trees have small height.
- Traversals: preorder, inorder, postorder (depth-first) and level order (breadth-first with a queue).
- In a BST, left subtrees are smaller and right subtrees larger; inorder traversal gives sorted order.
- BST operations cost $O(\text{height})$; sorted insertions create skewed trees.
- AVL and red-black trees rebalance with rotations to guarantee $O(\log n)$.
- B-trees, tries and segment trees solve specialised problems.

Practice Questions

1. Insert 50, 30, 70, 20, 40, 60, 80 into an empty BST and draw the result. Give its inorder and preorder traversals.
2. Delete 30 from your tree in question 1. Which case applies?
3. Why does inserting sorted values into a plain BST make it slow?

4. Write a recursive function that returns the number of leaves in a binary tree.
5. Explain what a trie is and why it is useful for autocomplete.

Heaps and Hash Tables

This chapter covers two structures that appear everywhere in practice. A **heap** always gives you the smallest (or largest) element quickly, which makes it the natural home for priority queues. A **hash table** finds, inserts and deletes items by key in expected constant time, making it perhaps the single most useful data structure in everyday programming.

Heaps

A **binary heap** is a **complete binary tree** (every level full except possibly the last, which fills from the left) that satisfies the **heap property**:

- In a **min-heap**, every node is smaller than or equal to its children, so the minimum is at the root.
- In a **max-heap**, every node is greater than or equal to its children, so the maximum is at the root.

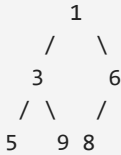
A heap is *not* sorted. It only guarantees that each parent beats its children, which is exactly enough to find the extreme element instantly.

Storing a Heap in an Array

Because the tree is complete, it can be stored in an array with no pointers at all. For the node at index i (0-based):

```
left child  = 2i + 1
right child = 2i + 2
parent      = (i - 1) // 2
```

The min-heap below is stored as [1, 3, 6, 5, 9, 8]:



Key idea: The complete-tree shape guarantees a height of about $\log n$, and the array layout means no pointer overhead. Both make heaps fast and compact.

Insertion: Sift Up

1. Place the new element at the end of the array (the next free spot in the bottom level).
2. While it is smaller than its parent (in a min-heap), swap it with the parent.

This takes $O(\log n)$, because the element rises at most the height of the tree.

Removing the Minimum: Sift Down

1. The minimum is at the root. Save it.
2. Move the last element to the root.

3. While it is larger than its smaller child, swap it with that child.

Also $O(\log n)$.

```
def sift_down(heap, i):
    n = len(heap)
    while True:
        smallest, l, r = i, 2 * i + 1, 2 * i + 2
        if l < n and heap[l] < heap[smallest]:
            smallest = l
        if r < n and heap[r] < heap[smallest]:
            smallest = r
        if smallest == i:
            return
        heap[i], heap[smallest] = heap[smallest], heap[i]
        i = smallest
```

Building a Heap in $O(n)$

Inserting n elements one by one costs $O(n \log n)$. **Heapify** does better: call sift-down on every non-leaf node, starting from the last one and moving backwards to the root. Most nodes are near the bottom and sift down only a little, so the total work is $O(n)$.

Heap Operations Summary

Operation	Time
Peek at min (or max)	$O(1)$
Insert	$O(\log n)$
Remove min (or max)	$O(\log n)$
Build from n elements	$O(n)$
Search for an arbitrary value	$O(n)$

Heaps in Python

Python's `heapq` module provides a min-heap on a plain list:

```
import heapq

tasks = []
heapq.heappush(tasks, (2, "write report"))
heapq.heappush(tasks, (1, "fix bug"))
heapq.heappush(tasks, (3, "reply to email"))
print(heapq.heappop(tasks))      # (1, 'fix bug')
```

For a max-heap, push negated priorities.

Applications of Heaps

- **Priority queues:** CPU scheduling, hospital triage, event simulations.
- **Heap sort:** build a max-heap, then repeatedly move the maximum to the end (Chapter 6).
- **Top-k problems:** to find the k largest of n items, keep a *min-heap of size k* ; for each new item, if it beats the smallest in the heap, replace it. Time $O(n \log k)$, which is far better than sorting everything when k is small.
- **Merging k sorted lists:** keep the current head of each list in a min-heap and repeatedly take the smallest.
- **Running median:** keep a max-heap for the lower half of the numbers and a min-heap for the upper half.

- **Graph algorithms:** Dijkstra's shortest paths and Prim's minimum spanning tree (Chapter 9).

Hash Tables

Suppose we want to store student records and look them up by roll number instantly. A **hash table** does this by computing an array index directly from the key.

Hash Functions

A **hash function** maps a key to an integer, which is reduced to an index in an array of **buckets**:

```
index = hash(key) % number_of_buckets
```

A good hash function:

- is **deterministic**: the same key always gives the same hash;
- is **fast** to compute;
- spreads keys **uniformly** across buckets, so similar keys land in different places.

For strings, a common approach is a polynomial rolling hash: treat characters as digits in some base and combine them, taking the remainder by a large number.

Collisions

Two different keys can hash to the same bucket. This is a **collision**, and it is unavoidable: there are far more possible keys than buckets. There are two main ways to handle collisions.

Separate Chaining

Each bucket holds a small list of all entries that hash there. To look up a key, hash it, then scan that bucket's list.

```
class HashMap:
    def __init__(self, size=8):
        self.buckets = [[] for _ in range(size)]

    def put(self, key, value):
        bucket = self.buckets[hash(key) % len(self.buckets)]
        for i, (k, _) in enumerate(bucket):
            if k == key:
                bucket[i] = (key, value)
                return
        bucket.append((key, value))

    def get(self, key):
        bucket = self.buckets[hash(key) % len(self.buckets)]
        for k, v in bucket:
            if k == key:
                return v
        raise KeyError(key)
```

Open Addressing

All entries live directly in the array. On a collision, **probe** for another empty slot:

- **Linear probing:** try the next slot, then the next (index + 1, + 2, ...). Simple and cache-friendly, but it causes **clustering**, long runs of filled slots.
- **Quadratic probing:** try index + 1, + 4, + 9, ... which reduces clustering.
- **Double hashing:** use a second hash function to decide the step size.

Deleting in open addressing needs care: mark slots as "deleted" (tombstones) rather than empty, or later searches may stop too early.

Load Factor and Resizing

The **load factor** is the number of entries divided by the number of buckets. As it grows, collisions become more frequent and operations slow down. When the load factor passes a threshold (often around 0.7), the table **resizes**: it allocates a bigger array, typically double the size, and **rehashes** every entry into it. Like dynamic arrays, the occasional $O(n)$ resize averages out to amortised $O(1)$.

Performance

Operation	Average	Worst case
Insert	$O(1)$	$O(n)$
Lookup	$O(1)$	$O(n)$
Delete	$O(1)$	$O(n)$

The worst case happens when many keys collide, for example with a poor hash function or deliberately chosen keys. Good hash functions and resizing keep the average case the one you actually see.

Key idea: Hash tables give up ordering to gain speed. They cannot efficiently answer "what is the smallest key?" or "list keys between 10 and 20". Use a balanced BST when you need order.

Sets and Maps

Python's `dict` and `set`, Java's `HashMap` and `HashSet`, and C++'s `unordered_map` are all hash tables. Keys must be **hashable**: they must not change while stored, which is why Python lists cannot be dictionary keys but tuples can.

Classic Hashing Problems

Two sum: find two numbers adding to a target in $O(n)$.

```
def two_sum(nums, target):
    seen = {}                                # value -> index
    for i, x in enumerate(nums):
        if target - x in seen:
            return seen[target - x], i
        seen[x] = i
```

Frequency counting: count words in a document with a dictionary, in $O(n)$.

Grouping anagrams: use the sorted letters of each word as a key: "eat", "tea" and "ate" all map to "aet".

Detecting duplicates and **caching** (memoization, Chapter 5) also rely on hash tables.

Beyond Basic Hash Tables

- **Cryptographic hashes** such as SHA-256 are designed so that collisions are practically impossible to find. They are used for passwords, digital signatures and file integrity checks, not for ordinary hash tables, because they are slower.
- **Bloom filters** use several hash functions and a bit array to test set membership with very little memory, allowing occasional false positives but never false negatives.
- **Consistent hashing** spreads data across many servers so that adding or removing a server moves only a small fraction of the keys.

Heap vs Hash Table vs Balanced BST

Need	Best structure
Repeatedly get the min or max	Heap
Look up by exact key	Hash table
Keep keys ordered, range queries	Balanced BST

Summary

- A binary heap is a complete tree with the heap property, stored compactly in an array.
- Insert and remove-min are $O(\log n)$ via sift-up and sift-down; peek is $O(1)$; heapify builds a heap in $O(n)$.
- Heaps implement priority queues and solve top-k, merging and running-median problems.
- Hash tables map keys to buckets with a hash function, giving average $O(1)$ operations.
- Collisions are handled by chaining or open addressing; resizing keeps the load factor low.
- Hash tables are unordered; choose a heap or a BST when order matters.

Practice Questions

1. Insert 7, 2, 9, 1, 5 into an empty min-heap and show the array after each insertion.
2. For the node at index 4 in an array heap, what are the indexes of its parent and children?
3. How would you find the 10 largest numbers in a stream of one million numbers efficiently?
4. Compare separate chaining and linear probing.
5. What is the load factor, and why do hash tables resize?

Graphs

A **graph** models relationships: people and friendships, cities and roads, web pages and links, courses and prerequisites. Trees and linked lists are special kinds of graphs. Once you can represent a problem as a graph, a large toolbox of algorithms becomes available: searching, shortest paths, spanning trees and ordering tasks.

Graph Vocabulary

A graph $G = (V, E)$ is a set of **vertices** (nodes) V and **edges** E connecting pairs of vertices.

- **Undirected graph:** edges have no direction, like a two-way road or a Facebook friendship.
- **Directed graph (digraph):** edges have a direction, like a one-way road or an Instagram follow.
- **Weighted graph:** each edge has a number (distance, cost, time).
- **Degree:** the number of edges touching a vertex. In a digraph, **in-degree** counts incoming edges and **out-degree** outgoing ones.
- **Path:** a sequence of vertices connected by edges. A **simple path** repeats no vertex.

- **Cycle:** a path that starts and ends at the same vertex.
- **Connected graph:** every vertex can reach every other (undirected).
- **Connected components:** the separate "islands" of a graph.
- **DAG:** a directed acyclic graph, a directed graph with no cycles. Course prerequisites form a DAG.
- **Dense vs sparse:** dense graphs have close to the maximum V^2 edges; sparse graphs have far fewer.

Representing Graphs

Adjacency Matrix

A $V \times V$ table where cell $[u][v]$ is 1 (or the weight) if there is an edge from u to v .

- Checking whether an edge exists: **$O(1)$** .
- Space: **$O(V^2)$** , wasteful for sparse graphs.
- Listing a vertex's neighbours: $O(V)$.

Adjacency List

Each vertex stores a list of its neighbours.

```
graph = {  
    "A": ["B", "C"],  
    "B": ["A", "D"],  
    "C": ["A", "D"],  
    "D": ["B", "C", "E"],  
    "E": ["D"],  
}
```

- Space: $O(V + E)$.
- Listing neighbours: $O(\text{degree})$.
- Checking one specific edge: $O(\text{degree})$.

Key idea: Most real-world graphs (roads, social networks, the web) are sparse, so adjacency lists are the usual choice. Use a matrix for small or dense graphs, or when you constantly ask "is there an edge between u and v ?".

Breadth-First Search (BFS)

BFS explores the graph in layers: first the start vertex, then all its neighbours, then their neighbours, and so on. It uses a **queue** and marks vertices as visited so each is processed once.

```

from collections import deque

def bfs(graph, start):
    dist = {start: 0}
    q = deque([start])
    while q:
        u = q.popleft()
        for v in graph[u]:
            if v not in dist:
                dist[v] = dist[u] + 1
                q.append(v)
    return dist # number of edges from
               # start to each reachable vertex

```

Time $O(V + E)$. Because BFS reaches vertices in order of distance, it finds **shortest paths in unweighted graphs**, measured in number of edges. Applications include "degrees of separation" in social networks, the fewest moves in a puzzle, and peer discovery in networks.

Depth-First Search (DFS)

DFS goes as deep as possible along one path before backtracking. It uses recursion (or an explicit **stack**).

```

def dfs(graph, u, visited=None):
    if visited is None:
        visited = set()
    visited.add(u)
    for v in graph[u]:
        if v not in visited:
            dfs(graph, v, visited)
    return visited

```

Time $O(V + E)$. DFS is the basis of many algorithms:

- **Connected components:** run DFS from each unvisited vertex; each run discovers one component.
- **Cycle detection:** in an undirected graph, finding an already visited vertex that is not your parent means a cycle. In a directed graph, reaching a vertex that is still "in progress" on the current path means a cycle.
- **Topological sorting** (below).
- **Maze solving and path finding** with backtracking.

	BFS	DFS
Data structure	Queue	Stack / recursion
Explores	Level by level	Deep first
Shortest path (unweighted)	Yes	No
Typical uses	Shortest steps, nearest items	Components, cycles, ordering

Topological Sorting

A **topological order** of a DAG lists vertices so that every edge goes from earlier to later: every course appears after its prerequisites.

Kahn's algorithm (BFS-based):

1. Compute the in-degree of every vertex.

2. Put all vertices with in-degree 0 into a queue.
3. Repeatedly remove a vertex, add it to the order, and decrease the in-degree of its neighbours; any neighbour reaching 0 joins the queue.
4. If the order contains fewer than V vertices, the graph has a cycle, and no valid order exists.

Topological sort is used in build systems (compile dependencies first), task scheduling and spreadsheet recalculation. It also appears in this very app: rooms in a Knowledge Dungeon unlock in prerequisite order.

Shortest Paths in Weighted Graphs

Dijkstra's Algorithm

Dijkstra's algorithm finds the shortest distance from a source to every vertex when all edge weights are **non-negative**.

1. Set the distance to the source as 0 and every other distance to infinity.
2. Repeatedly pick the unvisited vertex with the smallest known distance (using a min-heap).
3. **Relax** its edges: if going through it gives a neighbour a shorter distance, update that distance.

```

import heapq

def dijkstra(graph, src):
    # graph[u] = [(v,
    weight), ...]
    dist = {src: 0}
    heap = [(0, src)]
    while heap:
        d, u = heapq.heappop(heap)
        if d > dist[u]:
            continue # stale entry
        for v, w in graph[u]:
            nd = d + w
            if nd < dist.get(v, float("inf")):
                dist[v] = nd
                heapq.heappush(heap, (nd, v))
    return dist

```

Time **$O((V + E) \log V)$** with a binary heap. Maps and navigation apps use Dijkstra's algorithm and faster variants such as A*, which adds a heuristic estimate of the remaining distance.

Key idea: Dijkstra's algorithm fails with negative edge weights, because it assumes a vertex's distance is final once it is picked.

Bellman-Ford

Bellman-Ford relaxes every edge $V - 1$ times. It handles **negative weights** and can detect **negative cycles** (if an edge can still be relaxed after $V - 1$ rounds). It is slower: $O(V \times E)$.

Floyd-Warshall

Floyd-Warshall computes shortest paths between **all pairs** of vertices using dynamic programming, in $O(V^3)$. It is simple and suits small, dense graphs.

Minimum Spanning Trees

A **spanning tree** of a connected, undirected graph connects all vertices using exactly $V - 1$ edges and no cycles. A **minimum spanning tree (MST)** is one with the smallest total edge weight. Example: connect all the villages to the electricity network using the least total cable.

Kruskal's Algorithm

1. Sort all edges by weight.
2. Take edges from the smallest upwards, adding an edge only if it does not create a cycle.
3. Stop after $V - 1$ edges.

Cycle checks use the **union-find** (disjoint set) structure, which tracks which vertices are already connected. Time $O(E \log E)$.

Prim's Algorithm

1. Start from any vertex.
2. Repeatedly add the cheapest edge connecting the tree to a vertex not yet in it, using a min-heap.

Time $O(E \log V)$. Prim's suits dense graphs; Kruskal's is simple for sparse graphs.

Union-Find in Brief

Union-find supports two operations:

- **find(x)**: which group does x belong to?
- **union(x, y)**: merge the groups of x and y.

With **path compression** and **union by rank**, both run in nearly constant amortised time. Beyond Kruskal's algorithm, union-find answers questions like "are these two computers on the same network?".

Summary

- Graphs model relationships with vertices and edges, which may be directed and weighted.
- Adjacency lists use $O(V + E)$ space and suit sparse graphs; matrices give $O(1)$ edge checks.
- BFS uses a queue and finds shortest paths in unweighted graphs; DFS uses a stack and finds components, cycles and orderings.
- Topological sort orders a DAG so that dependencies come first.

- Dijkstra finds shortest paths with non-negative weights; Bellman-Ford handles negative weights; Floyd-Warshall solves all pairs.
- Kruskal's and Prim's algorithms build minimum spanning trees; union-find supports Kruskal's cycle checks.

Practice Questions

1. Draw the adjacency matrix and adjacency list for a graph of your choice with 5 vertices and 6 edges.
2. Run BFS from A on the example graph in this chapter and give the distance to each vertex.
3. Give a topological order for: Maths1 -> Maths2, Maths2 -> ML, Programming -> DSA, DSA -> ML.
4. Why does Dijkstra's algorithm fail with negative edge weights? Give a small example.
5. Run Kruskal's algorithm on a weighted graph of your choice and show which edges are chosen.

Greedy Algorithms and Dynamic Programming

Many problems ask for the *best* answer: the cheapest route, the most valuable set of items, the fewest coins. Trying every possibility is usually far too slow. This chapter covers two powerful strategies for optimisation problems. **Greedy algorithms** make the best-looking choice at each step and never look back. **Dynamic programming** breaks a problem into overlapping subproblems and remembers their answers.

Greedy Algorithms

A **greedy algorithm** builds a solution piece by piece, always choosing the option that looks best *right now*.

Greedy algorithms are simple and fast, but they only give the optimal answer for problems with two properties:

- **Greedy choice property:** a locally optimal choice can always be extended to a globally optimal solution.
- **Optimal substructure:** an optimal solution contains optimal solutions to its subproblems.

Key idea: A greedy algorithm must be *proved* correct for each problem. If you cannot argue why the greedy choice is safe, test it against a counterexample before trusting it.

Activity Selection

You have activities with start and finish times and one room. Schedule as many non-overlapping activities as possible.

Greedy rule: **always pick the activity that finishes earliest** among those that start after the last chosen activity ends.

```
def select_activities(activities):          # list of (start,
    finish)
    chosen, last_end = [], float("-inf")
    for start, finish in sorted(activities, key=lambda a:
        a[1]):
        if start >= last_end:
            chosen.append((start, finish))
            last_end = finish
    return chosen
```

Why it works: finishing early leaves the most room for everything else. Sorting dominates, so the time is $O(n \log n)$.

Fractional Knapsack

A thief can carry W kilograms and may take *fractions* of items (such as gold dust). Greedy rule: take items in order of **value per kilogram**, highest first, taking a fraction of the last one if needed. This is optimal.

Coin Change: When Greedy Fails

To pay an amount with the fewest coins, the greedy rule "always use the largest coin that fits" works for Indian currency (1, 2, 5, 10, 20, ...). But with coins {1, 3, 4} and amount 6, greedy takes $4 + 1 + 1$ (three coins), while $3 + 3$ (two coins) is better. Greedy is not always correct, and this problem needs dynamic programming in general.

Other Greedy Algorithms

- **Huffman coding:** builds optimal prefix codes for data compression by repeatedly merging the two least frequent symbols.
- **Dijkstra's algorithm, Prim's algorithm and Kruskal's algorithm** (Chapter 9) are all greedy.
- **Job sequencing with deadlines** and **minimum platforms at a railway station** are classic greedy exercises.

Dynamic Programming

Dynamic programming (DP) solves problems whose subproblems **overlap**: the same smaller problem appears again and again. Instead of solving it repeatedly, DP solves each subproblem once and stores the answer.

DP applies when a problem has:

1. **Optimal substructure:** the best solution is built from best solutions to subproblems.
2. **Overlapping subproblems:** the same subproblems recur.

Divide and conquer (like merge sort) also splits problems, but its subproblems are independent, so there is nothing to reuse.

Two Styles

Recall the naive Fibonacci from Chapter 5, which takes $O(2^n)$ time.

Top-down (memoization): keep the recursive structure, but cache results.

```
from functools import lru_cache

@lru_cache(maxsize=None)
def fib(n):
    return n if n < 2 else fib(n - 1) + fib(n - 2)
```

Bottom-up (tabulation): fill a table from the smallest subproblems upwards.

```
def fib(n):
    if n < 2:
        return n
    prev, cur = 0, 1
    for _ in range(n - 1):
        prev, cur = cur, prev + cur
    return cur
```

Both run in $O(n)$. The bottom-up version also avoids recursion and, here, needs only $O(1)$ space.

How to Design a DP Solution

1. **Define the state:** what does $dp[i]$ (or $dp[i][j]$) mean, in words?
2. **Write the recurrence:** how does a state depend on smaller states?
3. **Set the base cases.**
4. **Choose the order** of computation so that dependencies are computed first.
5. **Find the answer** in the table.
6. **Optimise space** if only the previous row or two are needed.

Classic DP Problems

Climbing Stairs

You can climb 1 or 2 steps at a time. How many ways are there to reach step n ?

- State: $ways[i]$ = number of ways to reach step i .
- Recurrence: $ways[i] = ways[i - 1] + ways[i - 2]$ (the last move was either 1 step or 2 steps).
- Base: $ways[0] = 1$, $ways[1] = 1$.

It is Fibonacci in disguise.

Coin Change (Minimum Coins)

```
def min_coins(coins, amount):
    INF = float("inf")
    dp = [0] + [INF] * amount          # dp[a] = fewest
    coins to make a
    for a in range(1, amount + 1):
        for c in coins:
            if c <= a and dp[a - c] + 1 < dp[a]:
                dp[a] = dp[a - c] + 1
    return dp[amount] if dp[amount] != INF else -1

min_coins([1, 3, 4], 6)    # 2 (3 + 3), where greedy gave 3
```

Time $O(\text{amount} \times \text{number of coins})$.

0/1 Knapsack

Each item has a weight and value; you either take it whole or leave it. Maximise value within capacity W .

- State: $dp[i][w]$ = best value using the first i items with capacity w .
- Recurrence: $dp[i][w] = \max(dp[i-1][w], \text{value}_i + dp[i-1][w - \text{weight}_i])$ if item i fits; otherwise $dp[i-1][w]$.

```
def knapsack(weights, values, W):
    dp = [0] * (W + 1)
    for wt, val in zip(weights, values):
        for w in range(W, wt - 1, -1):    # go backwards so
            each item is used once
            dp[w] = max(dp[w], val + dp[w - wt])
    return dp[W]
```

Time $O(n \times W)$, space $O(W)$.

	Fractional knapsack	0/1 knapsack
Can split items?	Yes	No
Correct method	Greedy by value/weight	Dynamic programming
Time	$O(n \log n)$	$O(n \times W)$

Longest Common Subsequence (LCS)

A **subsequence** keeps characters in order but may skip some. The LCS of "ABCBDAB" and "BDCABA" has length 4 (for example "BCBA").

- State: $dp[i][j]$ = LCS length of the first i characters of X and the first j of Y .
- Recurrence: if $X[i-1] == Y[j-1]$, $dp[i][j] = dp[i-1][j-1] + 1$; otherwise $dp[i][j] = \max(dp[i-1][j], dp[i][j-1])$.

```
def lcs(x, y):
    dp = [[0] * (len(y) + 1) for _ in range(len(x) + 1)]
    for i in range(1, len(x) + 1):
        for j in range(1, len(y) + 1):
            if x[i - 1] == y[j - 1]:
                dp[i][j] = dp[i - 1][j - 1] + 1
            else:
                dp[i][j] = max(dp[i - 1][j], dp[i][j - 1])
    return dp[-1][-1]
```

Time $O(m \times n)$. LCS underlies `diff` tools and DNA sequence comparison.

Edit Distance

The **edit distance** between two words is the minimum number of insertions, deletions and substitutions needed to turn one into the other ("kitten" to "sitting" needs 3). Its DP table looks just like LCS. Spell checkers use it to suggest corrections.

Longest Increasing Subsequence (LIS)

Find the longest subsequence in which values strictly increase. A simple DP is $O(n^2)$: $dp[i]$ is the length of the longest increasing subsequence ending at i . A clever method with binary search achieves $O(n \log n)$.

Key idea: Most DP problems follow a few patterns: 1-D sequences (stairs, LIS), two strings (LCS, edit distance), choose-or-skip items (knapsack) and grids (unique paths). Recognising the pattern is half the solution.

Greedy vs Dynamic Programming

	Greedy	Dynamic programming
Decisions	One choice per step, never revisited	Considers all choices via subproblems
Speed	Usually faster	Usually slower, uses more memory
Correctness	Only for problems with the greedy choice property	Whenever there is optimal substructure
Examples	Activity selection, Huffman, Dijkstra	Knapsack, LCS, edit distance, coin change

A good strategy: try to find a greedy rule and a counterexample to it. If you find a counterexample, switch to DP.

Summary

- Greedy algorithms take the best local choice at each step; they are fast but must be proved correct.
- Activity selection, fractional knapsack, Huffman coding, Dijkstra, Prim and Kruskal are greedy.
- Dynamic programming reuses answers to overlapping subproblems, top-down with memoization or bottom-up with tables.
- Design DP by defining the state, the recurrence, the base cases and the computation order.

- Classic DP problems include climbing stairs, coin change, 0/1 knapsack, LCS, edit distance and LIS.
- When a greedy rule has a counterexample, dynamic programming is often the answer.

Practice Questions

1. Apply activity selection to (1,4), (3,5), (0,6), (5,7), (3,9), (5,9), (6,10), (8,11).
2. Show that greedy coin change fails for coins {1, 5, 6, 9} and amount 11.
3. Fill the DP table for the 0/1 knapsack with weights [1, 3, 4], values [15, 20, 30] and capacity 4.
4. Compute the LCS length of "AGGTAB" and "GXTXAYB".
5. Explain the difference between memoization and tabulation.